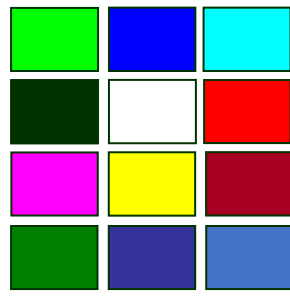


Elements of RT Level circuits: RTL Design Methodology

❖ Designing RTL IP Cores

- IP Cores
- Filter Concepts
- FIR Filter

Filter Concepts



Filters

Are crucial tools in signal processing.

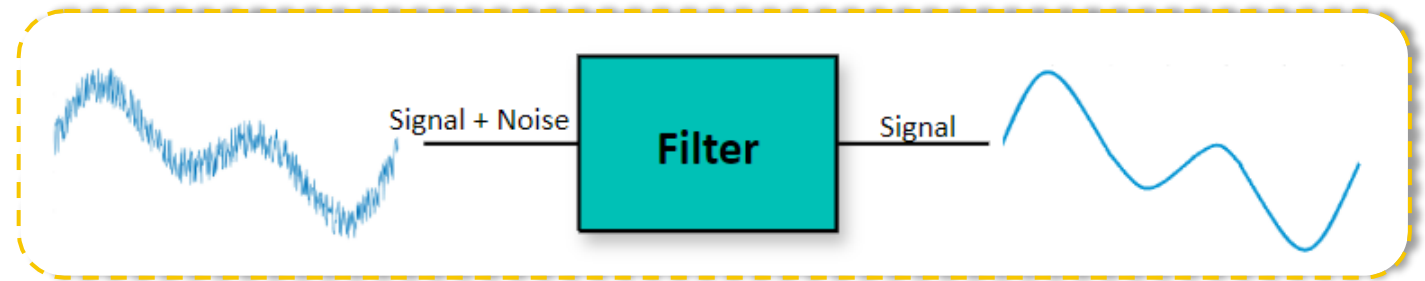
Designed to manipulate signals to achieve a desired effect.

- **General Functions:**

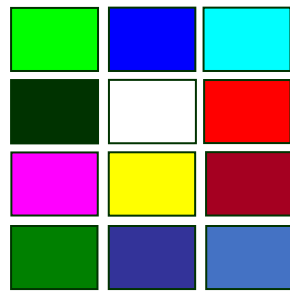
- Remove noise
- Extract useful information
- Smooth/ Sharpen signals
- Modify signal characteristics

- **Applications**

- Audio/ Image Processing
- Communication Systems
- Biomedical Signal Processing
- Data Analysis

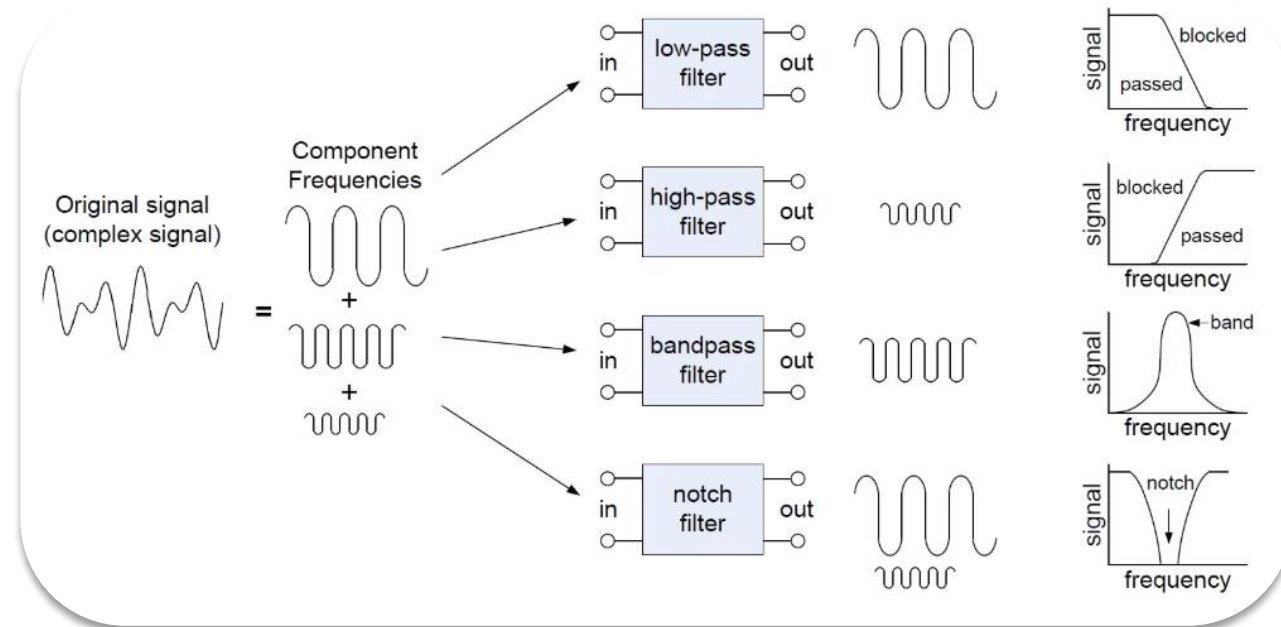


Filter Concepts

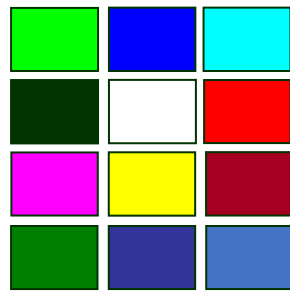


Types of filters

- **Low-pass filter:** Allows low-frequency signals.
- **High-pass filter:** Allows high-frequency signals.
- **Band-pass filter:** Allows a specific range of frequencies.
- **Band-stop (Notch) filter:** Attenuates a specific range of frequencies while allowing frequencies outside this range to pass through.

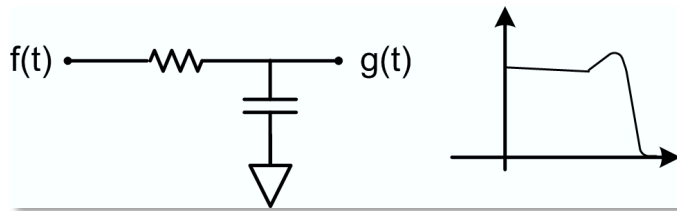


Filter Concepts



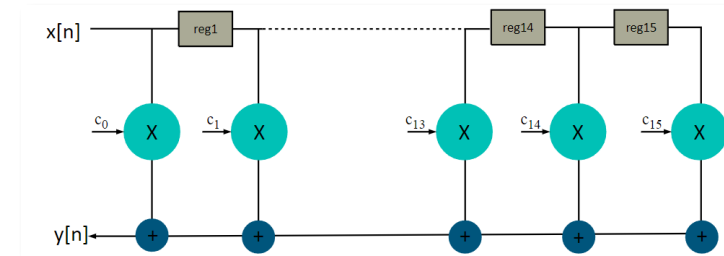
Analog filters

- Are used for filtering analog signals.
- Designed with components like resistor, inductor, and capacitor.
- Sensitive to environmental changes.



✓ Digital filters

- Are used for filtering digital sequences.
- Designed with digital components like FF, counter, shift registers, ALU, and etc.
- Less sensitive to environmental changes, noise, and disturbances.
- The simplicity and stability make them ideal for many digital signal processing applications.



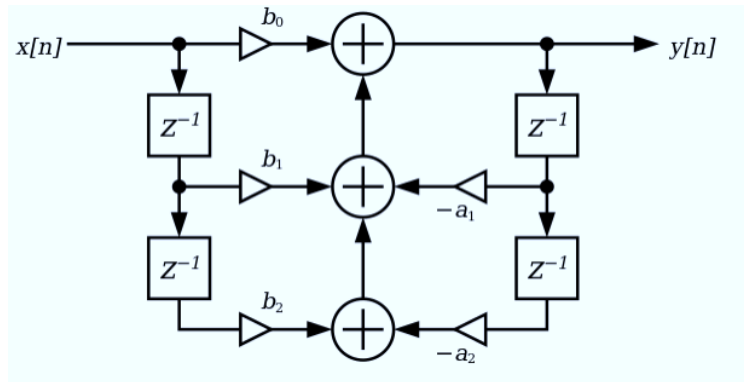
Filter Concepts

Digital filters

Infinite Impulse Response (IIR)

- Are chosen where memory is constrained and linear phase is not as critical.

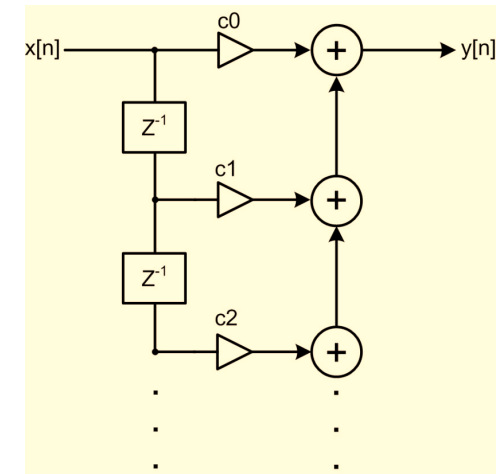
$$y[n] = -\sum a[j] * y[n-j] + \sum b[k] * x[n-k]$$



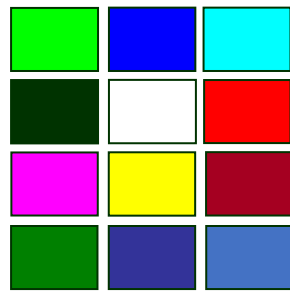
Finite Impulse Response (FIR)

Are used where linear phase is crucial and a respectable amount of memory and computational performance are available.

$$y[n] = \sum c[k] * x[n-k]$$



Basic Concepts



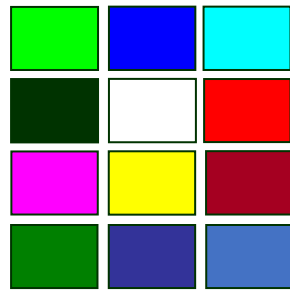
FIR Filter characteristics

- **Order:**
 - The highest power of the delay element.
 - The number of previous inputs and outputs it considers in its defining equation.
- **Coefficient:**
 - Determines the weights applied to the input signal samples.

$$y[n] = \sum_{k=0}^N c[k] * x[n-k] = c_0 x[n] + c_1 x[n-1] + \dots + c_N x[n-N]$$

Diagram illustrating the FIR filter equation with annotations:

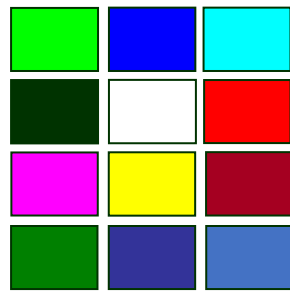
- Filter order:** Indicated by a yellow circle around the summation limit N .
- Coefficient:** Indicated by a yellow circle around $c[k]$.
- output signal at time n :** Indicated by a blue circle around $y[n]$.
- Input signal at time n :** Indicated by a blue circle around $x[n]$.



Elements of RT Level circuits: RTL Design Methodology

❖ Designing RTL IP Cores

- IP Cores
- Filter Concepts
- FIR Filter



FIR Filter

Functionality

- Applies a fixed set of coefficients to past input values when processing signals.
- Each output sample is calculated as a weighted sum of the current and previous input samples.
- This method is repeated until all inputs are fully processed.
- **Example:** A second order FIR filter with following input and coefficient.

$$x = [1, 2, 3, 4]$$

$$c = [0.3, 0.4, 0.25]$$

$$y[n] = \sum_{k=0}^2 c[k] * x[n-k] = c_0 x[n] + c_1 x[n-1] + c_2 x[n-2]$$

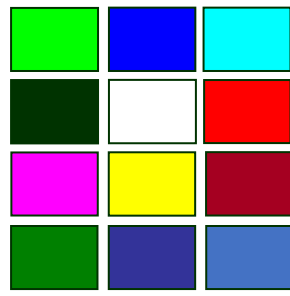
$$n=0 \rightarrow y[0] = 0.3 * 1 + 0.4 * 0 + 0.25 * 0 = 0.3$$

$$n=1 \rightarrow y[1] = 0.3 * 2 + 0.4 * 1 + 0.25 * 0 = 0.85$$

$$n=2 \rightarrow y[2] = 0.3 * 3 + 0.4 * 2 + 0.25 * 1 = 2.85$$

$$n=3 \rightarrow y[3] = 0.3 * 4 + 0.4 * 3 + 0.25 * 2 = 2.9$$

FIR Filter

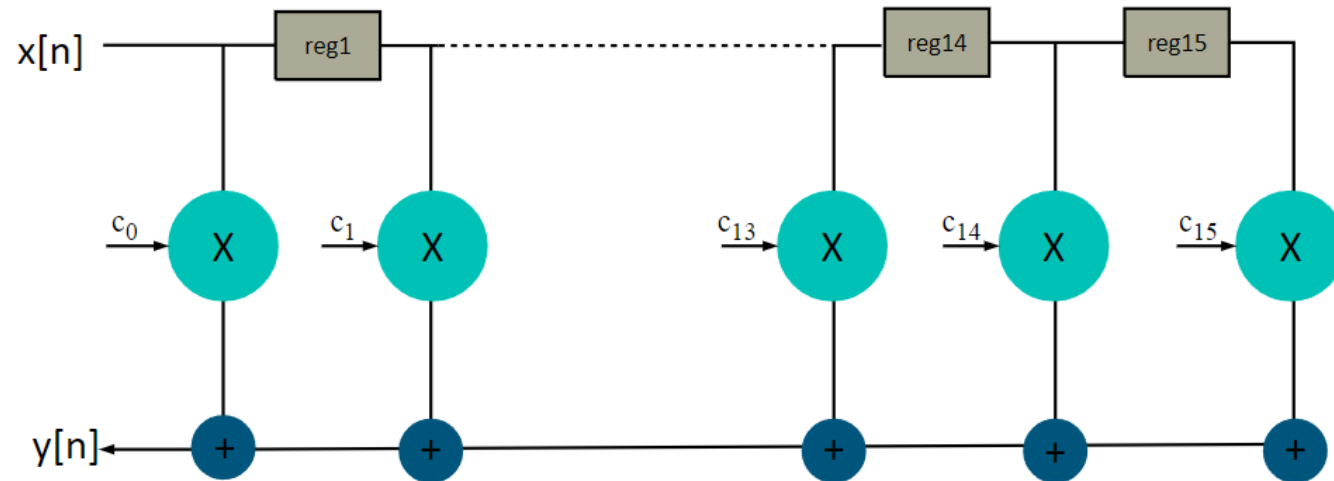


Hardware Design

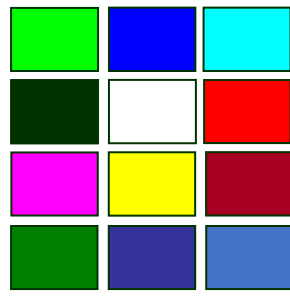
- Applies a fixed set of coefficients to past input values when processing signals.
- Each output sample is calculated as a weighted sum of the current and previous input samples.
- This method is repeated until all inputs are fully processed.

$$y[n] = \sum_{k=0}^{15} c[k] * x[n-k] = c_0 x[n] + c_1 x[n-1] + \dots + c_{15} x[n-15]$$

Spatial implementation



FIR Filter

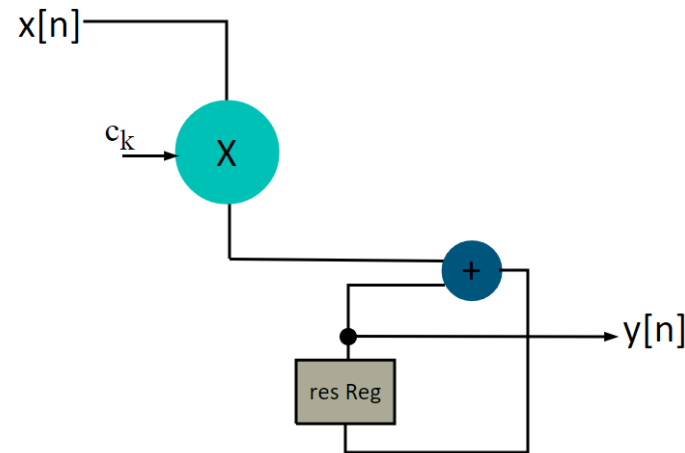


Hardware Design

- Applies a fixed set of coefficients to past input values when processing signals.
- Each output sample is calculated as a weighted sum of the current and previous input samples.
- This method is repeated until all inputs are fully processed.

$$y[n] = \sum_{k=0}^{15} c[k] * x[n-k] = c_0 x[n] + c_1 x[n-1] + \dots + c_{15} x[n-15]$$

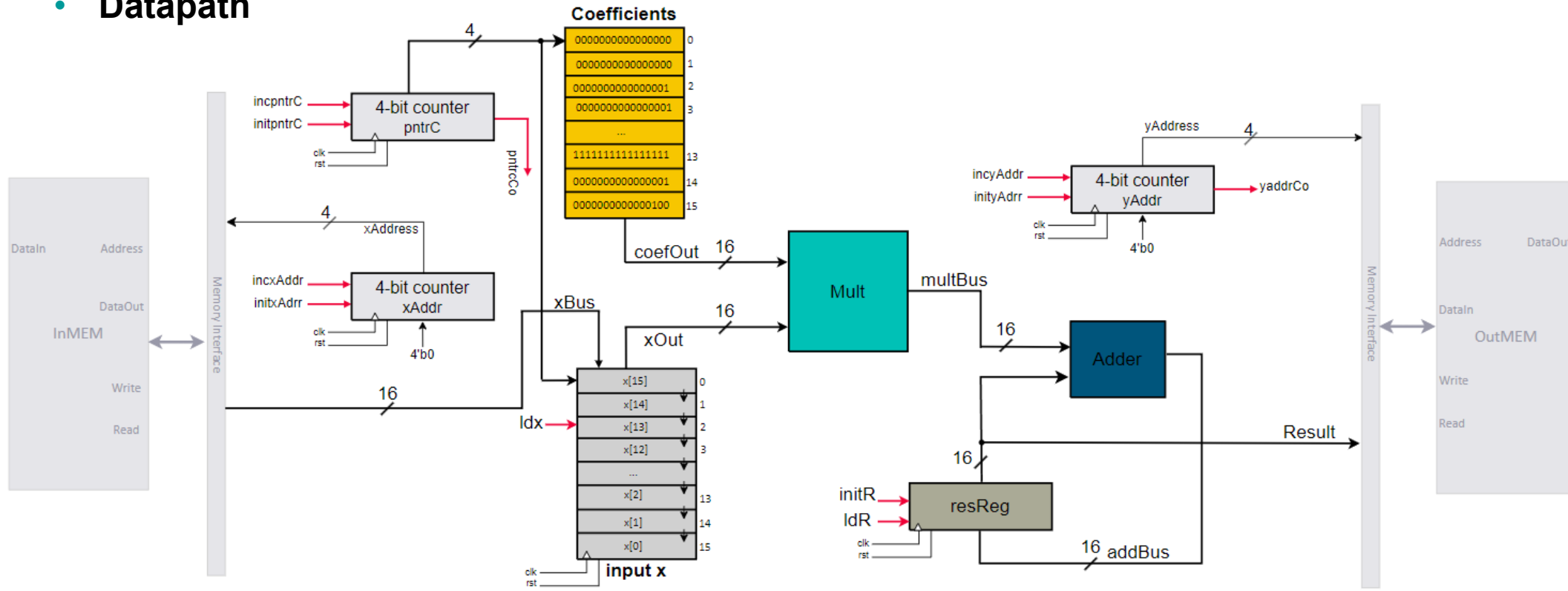
Temporal implementation



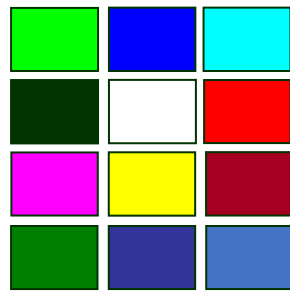
FIR Filter

Hardware Design

- Temporal Implementation
 - Datapath

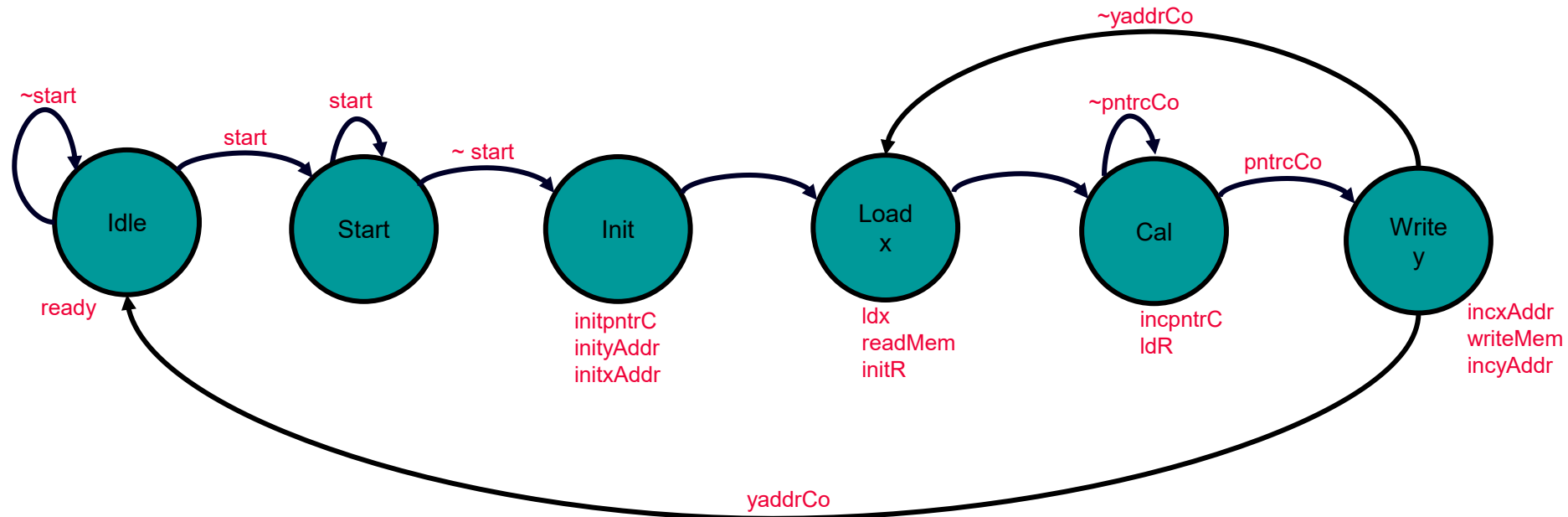


FIR Filter



Hardware Design

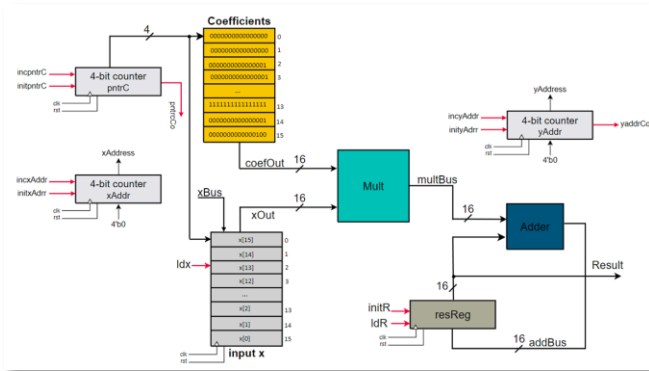
- **Controller**
- Applies a fixed set of coefficients to past input values when processing signals.
- Each output sample is **calculated** as a weighted sum of the current and previous input samples.
- This method is **repeated** until all inputs are fully processed.



FIR Filter

Temporal Implementation

- SystemVerilog Code
- Datapath

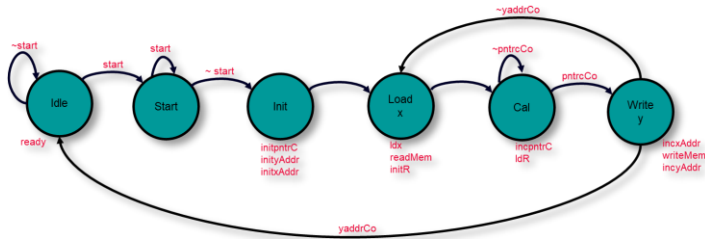


```
1 module Datapath_FIR (clk, rst, xBus, Result, xAddress, yAddress, initR, ldR, ldC, ldD, incpntrC,
2   initpntrC, inityAddr, initxAddr, pntrCo, yAddrCo, incxAddr, incyAddr);
3   input  clk, rst;
4   input  [15:0] xBus;
5   output [15:0] Result;
6   output [3:0]  xAddress;
7   output [3:0]  yAddress;
8   input  initR, ldC, ldR, ldD, initxAddr, incxAddr, incpntrC, initpntrC, inityAddr, incyAddr;
9   output pntrCo, yAddrCo;
10
11   wire signed [15:0] addBus;
12   wire signed [15:0] coefOut, xOut;
13   wire signed [15:0] multBus;
14   wire [3:0] PntrC;
15
16   LUT coef (.address(PntrC), .data(coefOut));
17
18   ShRegFile xinput (.clk(clk), .rst(rst), .write_en(ldx), .write_data(xBus), .read_addr(PntrC), .read_data(xOut));
19
20   multiplier mult (.A(coefOut), .B(xOut), .out(multBus));
21
22   Adder add (.A(multBus), .B(Result), .out(addBus));
23
24   Counter #(4) pntrc (.clk(clk), .rst(rst), .initCounter(initpntrC), .incCounter(incpntrC), .PI(4'b0), .PO(PntrC), .Co(pntrCo));
25
26   Counter #(4) xaddr (.clk(clk), .rst(rst), .initCounter(initxAddr), .incCounter(incxAddr), .PI(4'b0), .PO(xAddress), .Co());
27
28   Counter #(4) yaddr (.clk(clk), .rst(rst), .initCounter(inityAddr), .incCounter(incyAddr), .PI(4'b0), .PO(yAddress), .Co(yAddrCo));
29
30   Register #(16) resReg (.clk(clk), .rst(rst), .init(initR), .load(ldR), .inReg(addBus), .outReg(Result));
31
32 endmodule
```

FIR Filter

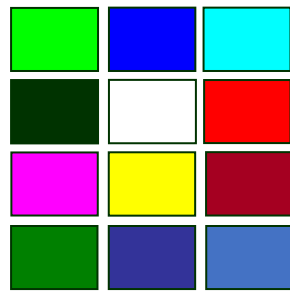
Temporal Implementation

- SystemVerilog Code
- Controller



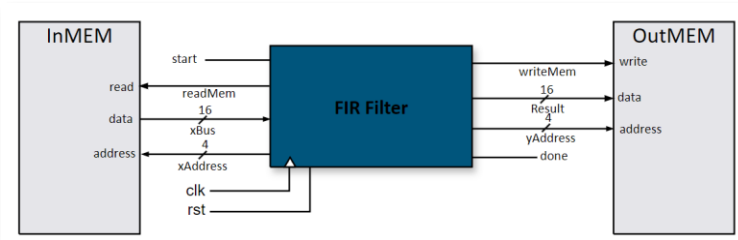
```
1 module Controller_FIR (clk, rst, start, done, initR, ldX, ldC, ldR, ldD, incpntnC, initpntnC, inityAddr,
2                       initxAddr, pntroCo, yAddrCo, incxAddr, incyAddr, readMem, writeMem);
3
4   input  clk, rst, start;
5   input  yAddrCo, pntroCo;
6   output logic readMem, writeMem;
7   output logic done, initR, ldX, ldC, ldR, ldD, incpntnC, initpntnC, inityAddr, initxAddr, incxAddr, incyAddr ;
8
9   parameter [2:0] Idle = 0, Start = 1, Init = 2, Loadx = 3, Cal = 4, Writey = 5;
10  logic [2:0] pstate, nstate;
11
12  always @(posedge clk, posedge rst) begin
13      if (rst)
14          pstate <= Idle;
15      else
16          pstate <= nstate;
17  end
18
19  always @(pstate, start, pntroCo, yAddrCo) begin
20
21      done = 1'b0;
22      ldX = 1'b0; ldC = 1'b0; ldR = 1'b0; ldD = 1'b0; initpntnC = 1'b0;
23      inityAddr = 1'b0; initxAddr = 1'b0; incxAddr = 1'b0; incyAddr = 1'b0;
24      readMem = 1'b0; writeMem = 1'b0; initR = 1'b0;
25
26      case (pstate)
27          Idle : begin
28              nstate = start ? Start : Idle;
29              done = 1;
30          end
31
32          Start : begin
33              nstate = start ? Start : Init;
34          end
35
36          Init : begin
37              nstate = Loadx;
38              initxAddr = 1;
39              inityAddr = 1;
40              initpntnC = 1;
41          end
42
43          Loadx : begin
44              nstate = Cal;
45              readMem = 1;
46              ldX = 1;
47              initR = 1;
48          end
49
50          Cal : begin
51              nstate = pntroCo ? Writey : Cal;
52              incpntnC = 1;
53              ldR = 1;
54          end
55
56          Writey : begin
57              nstate = yAddrCo ? Idle : Loadx;
58              writeMem = 1;
59              incyAddr = 1;
60              incxAddr = 1;
61          end
62
63          default: nstate = Idle;
64      endcase
65  end
66
67 endmodule
```

FIR Filter



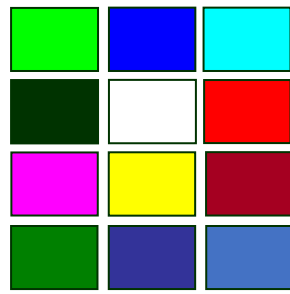
Temporal Implementation

- SystemVerilog Code
- TopLevel



```
1 module FIRFilter (clk, rst, start, done, readMem, writeMem, xAddress, yAddress, xBus, Result);
2     input clk, rst;
3     input start;
4     input signed [15:0] xBus;
5     output signed [15:0] Result;
6     output [3:0] xAddress;
7     output [3:0] yAddress;
8     output readMem, writeMem, done;
9
10    wire initR, ldX, ldC, ldR, ldD;
11    wire initpntnC, inityAddr, initxAddr, pntnC, yAddrCo, incxAddr, incyAddr;
12
13    Controller_FIR cu(.clk(clk), .rst(rst), .start(start), .done(done), .initR(initR), .ldX(ldX),
14                    .ldC(ldC), .ldR(ldR), .ldD (ldD), .incpntnC(incpntnC), .initpntnC(initpntnC),
15                    .inityAddr(inityAddr), .initxAddr(initxAddr), .pntnC(pntnC), .yAddrCo(yAddrCo),
16                    .incxAddr(incxAddr), .incyAddr(incyAddr), .readMem(readMem), .writeMem(writeMem));
17
18    Datapath_FIR dp(.clk(clk), .rst(rst), .xBus(xBus), .Result(Result), .xAddress(xAddress), .yAddress(yAddress),
19                  .initR(initR), .ldX(ldX), .ldC(ldC), .ldR(ldR), .ldD (ldD), .incpntnC(incpntnC), .initpntnC(initpntnC),
20                  .inityAddr(inityAddr), .initxAddr(initxAddr), .pntnC(pntnC), .yAddrCo(yAddrCo), .incxAddr(incxAddr),
21                  .incyAddr(incyAddr));
22
23 endmodule
```

FIR Filter



Temporal Implementation

- SystemVerilog Code
- Simulation Result

```
1 timescale 1 ns / 1ns
2 module FIR_TB();
3
4     logic clk = 0, rst = 0, start = 0;
5     logic [15:0] xBus;
6     wire done;
7     wire readMem, writeMem;
8     wire [15:0] Result;
9     wire [3:0] yAddress, xAddress;
10
11     FIRFilter IP(.clk(clk), .rst(rst), .start(start), .done(done), .readMem(readMem), .writeMem(writeMem),
12                 .xAddress(xAddress), .yAddress(yAddress), .xBus(xBus), .Result(Result));
13
14     Memory #(16) xmem (.clk(clk), .rst(rst), .readMem(readMem), .writeMem(1'b0), .addrBus(xAddress), .inBus(), .outBus(xBus));
15     Memory #(16) ymem (.clk(clk), .rst(rst), .readMem(1'b0), .writeMem(writeMem), .addrBus(yAddress), .inBus(Result), .outBus());
16
17     always #5 clk = ~clk;
18     // Resetting
19     initial begin
20         #7 rst = 1'b1;
21         #26 rst = 1'b0;
22         @(negedge clk);
23     end
24
25     // Applying inputs
26     initial begin
27         #25 start = 1'b1;
28         #24 start = 1'b0;
29     end
30
31
32 endmodule
```

In Memory (x[n])

1	0100000000000000
2	1100000000000000
3	1100000000000000
4	1100000000000000
5	0100000000000000
6	0100000000000000
7	0100000000000000
8	0100000000000000
9	1100000000000000
10	1100000000000000
11	1100000000000000
12	1100000000000000
13	0100000000000000
14	0100000000000000
15	0100000000000000
16	1100000000000000

Out Memory (y[n])

1	0000000000000000
2	0000000000000000
3	0100000000000000
4	0000000000000000
5	0000000000000000
6	1000000000000000
7	1100000000000000
8	0100000000000000
9	1000000000000000
10	1100000000000000
11	1100000000000000
12	1000000000000000
13	1100000000000000
14	1000000000000000
15	0100000000000000
16	1100000000000000

0
0
16384

$$n=0 \rightarrow y[0] = 0 * 16384 = 0$$

$$n=1 \rightarrow y[1] = 0 * 16384 + 0 * -16384 = 0$$

$$n=2 \rightarrow y[2] = 0 * -16384 + 0 * -16384 + 1 * 16384 = 16384$$